

Assignment 1. Deep Learning in Data Science.

Author:
Rodrigo Castellano Ontiveros
roco@kth.se

Exercise 1

I successfully managed to write the functions to correctly compute the gradient analytically. The tests that I run consisted of calculating the relative error between a numerically computed gradient value and an analytically computed gradient value. For two different functions that calculate analytical gradient value, the relative errors for W and b were around 10^{-7} and for the slower but more accurate version 10^{-10} .

For each setting the obtained results for each setting of parameters are shown in figure [5]. The settings are the following:

1. $\lambda=0$, n epochs=40, n batch=100, $\eta=.1$ - Test accuracy: 28.2%.
2. $\lambda=0$, n epochs=40, n batch=100, $\eta=.001$ - Test accuracy: 38.9%.
3. $\lambda=.1$, n epochs=40, n batch=100, $\eta=.001$ - Test accuracy: 39.2%.
4. $\lambda=1$, n epochs=40, n batch=100, $\eta=.001$ - Test accuracy: 37.8%.

The effect of regularization is that the model avoids overfitting, which is a gap between accuracy of training and validation. In this case the model suffers from overfitting, so regularization is needed.

Regarding the value of η , the results are better as η decreases, but if it decreases too much the learning is too slow and we may never reach a local minimum. This means that η had to decrease so that the model could find narrower local minima. We can see in the plot of configuration 1 that the curve is not smooth, due to the high value of the learning rate.

For a lower learning rate, we try to change the regularization. In configuration 3, we see how the cost validation keeps decreasing, whereas the loss validation does not, thanks to L_2 regularization. In configuration 4, there is a high value of λ , which leads to higher values in the cost function. It is noticeable how the initial values of the cost function are very high, whereas they should be approximately $-\log(1/K) = -\log(1/10) = 2.3$, as in configuration 3.

Images representing the learnt weight matrix after the completion of training are shown in figure 10. In general, as accuracy improves, the image of the weights looks less like random points, even though the interesting point is that even though for the configuration 4 the accuracy is lower (the model has learn worse than configuration 3), the weights

has a shape more defined. This may be due to the use of regularization, allowing the model to generalize better.

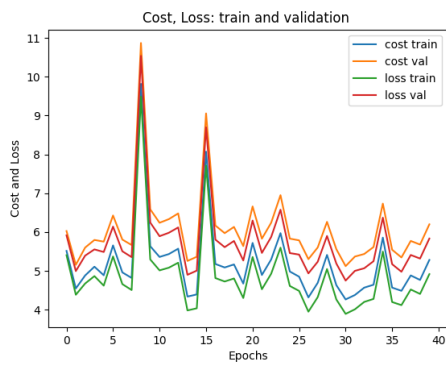


Figure 1: Configuration 1

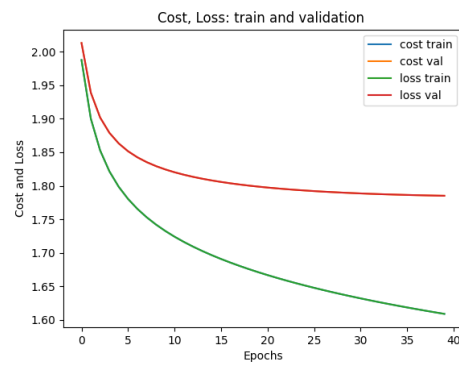


Figure 2: Configuration 2

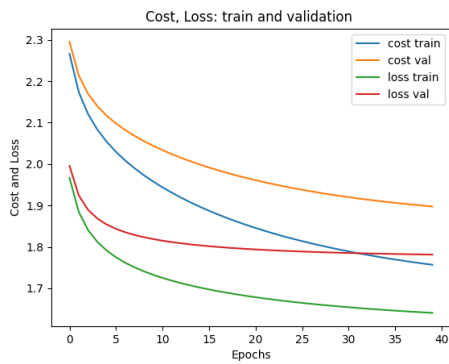


Figure 3: Configuration 3

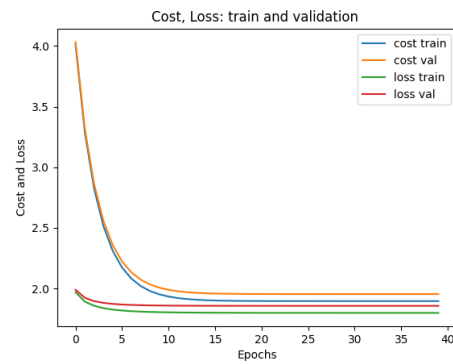


Figure 4: Configuration 4

Figure 5: Results for the cost and loss functions.

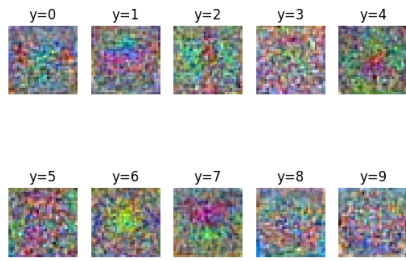


Figure 6: Configuration 1

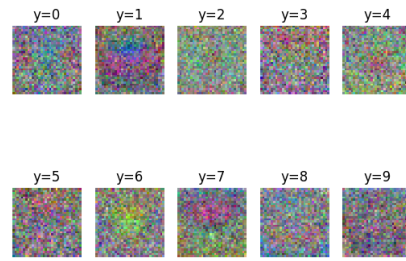


Figure 7: Configuration 2

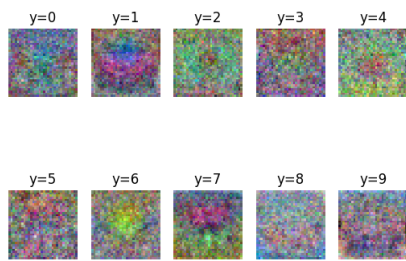


Figure 8: Configuration 3

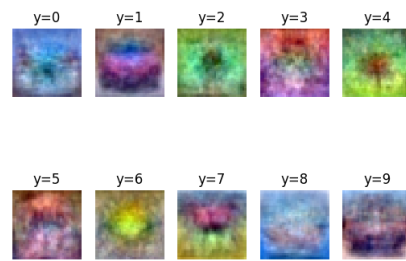


Figure 9: Configuration 4

Figure 10: Obtained images for the weights.

Exercise 2

1. Improve performance of the network

The parameters that were explored with grid search were: L_2 regularization (values 1, 0.1, 0.001), learning rate (0.001, 0.0001) and batch size (100,50). The best performance, 39.7% accuracy, was obtained with $\{n_epochs : 40, n_batch : 100, lambda : 0.1, eta : 0.001\}$. It seems that decreasing the learning rate gave the best results. Besides, the batch size did not have great impact on the performance. The regularization gave the best result with the value 0.1, given that the network suffers from overfitting and regularization is needed but not too high, because it would lead to underfitting. Maybe, with a even lower learning rate and more epochs, better results could be achieved.

Random flip of the images was implemented. Even the performance did not improve, it can be seen in figure [11] how the weights have changed and now are more symmetric.

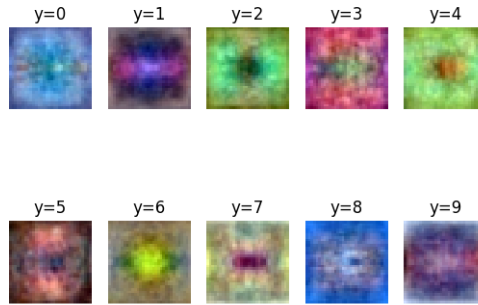


Figure 11: Weights from the training were images are flipped.

Finally, the learning rate was reduced by a factor of 10 every 30 epochs and the model was trained for 100 epochs (the initial value chosen for the learning rate was 0.1). The accuracy did improve, reaching 40.1% accuracy. As mentioned in the lecture, in the accuracy plot the difference can be appreciated when the learning rate is reduced, as shown in figure [14].

2. Train network - multiple binary cross-entropy (bce) losses

For this case, the derivative of the bce with respect to s turns out to be the same as with the previous loss function, the only change is that it is divided by K

$$\frac{\partial l}{\partial s} = \frac{1}{K}(Y - P)$$

For the cross-entropy loss, the test accuracy for $\{n_epochs : 40, n_batch : 100, lambda : 0.1\}$ and learning rate decrease every 30 epochs with initial value of 0.001 is 40.1%. For the bce loss, the test accuracy with the same parameters but higher initial learning rate (0.01)

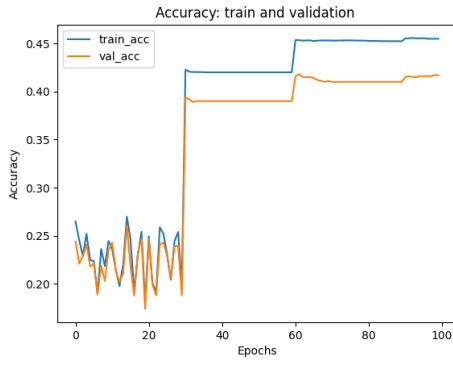


Figure 12: Accuracy of train and validation sets.

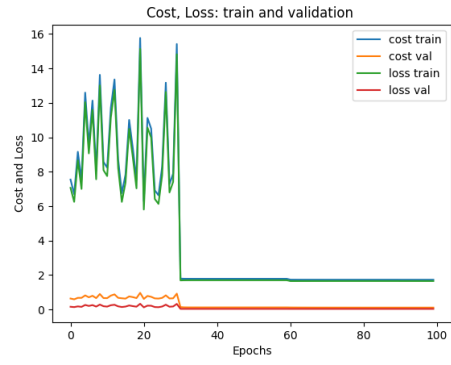


Figure 13: Cost and loss function of train and validation sets.

Figure 14: Obtained results with learning rate scheduled.

is 37.9%. This leads to the conclusion that the bce loss gives worse accuracy, but learns better to generalize, and this can be appreciated in the plot of the weights.

The histogram with the probability for the ground truth class for the examples correctly and those incorrectly classified can be found in figure [17]. In the histograms it is shown how confident the model is when it does a correct and incorrect prediction. In both cases, the model is more confident when the prediction is correct, although for the bce loss function the probability is generally lower than for cross-entropy loss.

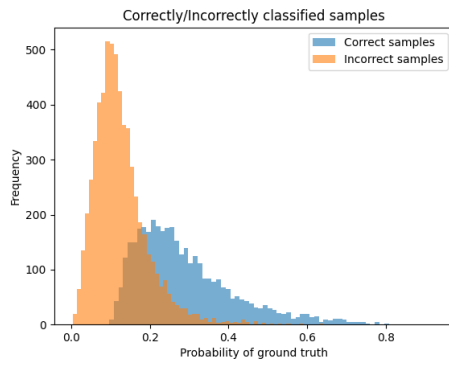


Figure 15: bce loss function

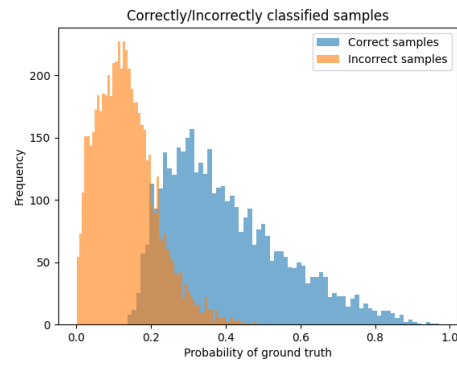


Figure 16: cross-entropy loss function

Figure 17: Obtained histograms for different loss functions.